

Reviewer Pack — Tensor-Amplified Asdim Forces Linear Communication Blow-up o...

Entry 916caa07c776 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 10:39:43 UTC

Tensor-Amplified Asdim Forces Linear Communication Blow-up on Inner-Product Gadgets

Entry ID: 916caa07c776 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 10:39:43 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Lifting (CGL) / Coarse Geometry & Asymptotic Dimension (geometric group theory applied to communication complexity) **Field B** (complexity object): Deterministic two-party communication complexity of lifted functions $f \circ G^n$, with G obtained by k -fold GadgetComposition of a base inner-product gadget IP_2

Statement:

Let IP_2 be the 2-bit inner-product MetricGadget on $X=Y=\{0,1\}^2$ with Hamming metric d , and let $G_k = IP_2^{\otimes k}$ under coordinatewise additive metric. For every Boolean $f:\{0,1\}^n \rightarrow \{0,1\}$ with deterministic query complexity $Q(f)$, the CoarseLiftingComplexity satisfies $CLC(f, G_k) \geq Q(f) \cdot \log_2(k+1) - O(n)$, and consequently $CC(f \circ G_k^n) \geq Q(f) \cdot \log_2(k+1) - O(n)$. Equivalently, $AsdimCertify(LiftedInputSpace(G_k), R=\text{diam}(G_k))$ returns multiplicity $m \geq k+1$, so by axiom A2 every deterministic protocol Π must have cost $\geq \log_2(k+1)$ per query bit recovered.

Rationale (proposer's reasoning):

This sub-conjecture directly tests axiom A3 (Asdim Amplification) chained with A2 (Protocol→Cover). A3 predicts that iteratively tensoring IP_2 (which already has $\text{asdim} \geq 1$ because its 16-point input space contains a metric line of length 4) yields G_k with $\text{asdim} \geq k - O(1)$. A2 then converts the cover multiplicity into a communication lower bound. If the conjecture holds, it provides a concrete constructive route to lifting bounds that scale with the tensor depth k , recovering Raz-McKenzie-style log-factor blow-ups from a purely coarse-geometric witness rather than from gadget-specific combinatorics. Failure would falsify A3 in the small-multiplicity regime and force a refinement of how asdim composes under $\otimes$.

Taxonomy category: LIFTING (status at proposal time:)

Framework membership: framework fw_6997a27304, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 98db1b6cf2d582ea

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 6 (k,n) cells with $k \in \{1,2,3\}$, $n \in \{2,3\}$, for both $f=\text{AND}_n$ and $f=\text{PARITY}_n$ (12 instances total, 3 seeds each for randomized greedy AsdimCertify tie-breaking): (a) certified multiplicity $m \geq k+1$, and (b) no enumerated protocol of cost $\leq \lceil n \cdot \log_2(k+1) \rceil - 1$ computes $f \circ G_k^n$. Support requires $\geq 10/12$ instances pass on $\geq 2/3$ seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - lifting theorem communication complexity inner product gadget deterministic query - asymptotic dimension coarse geometry communication complexity lower bounds - gadget composition tensor inner product communication complexity blow-up

Top relevant hits considered: - [<http://arxiv.org/abs/1904.13056v4>] Query-to-Communication Lifting Using Low-Discrepancy Gadgets - [<http://arxiv.org/abs/1104.4468v3>] A strong direct product theorem for quantum query complexity - [<http://arxiv.org/abs/2511.17227v3>] A Lifting Theorem for Hybrid Classical-Quantum Communication Complexity - [<http://arxiv.org/abs/cs/0111062v2>] One-way communication complexity and the Neciporuk lower bound on formula size - [<http://arxiv.org/abs/1110.6876>] Lower bounds for topological complexity - [<http://arxiv.org/abs/1608.06580v2>] Communication complexity of approximate Nash equilibria - [<http://arxiv.org/abs/0911.1773v3>] Perverse coherent sheaves on blow-up. III. Blow-up formula from wall-crossing - [<http://arxiv.org/abs/1607.03608v3>] On the tensor product of linear sites and Grothendieck categories

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json

# Helper functions for basic linear algebra and combinatorics
def dot_product(v1, v2):
    return sum(x * y for x, y in zip(v1, v2))

def hamming_distance(v1, v2):
    return sum(1 for x, y in zip(v1, v2) if x != y)
```

```

def matrix_multiply(A, B):
    n = len(A)
    result = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                result[i][j] += A[i][k] * B[k][j]
    return result

def gaussian_elimination(matrix, b):
    n = len(matrix)
    augmented_matrix = [row + [b[i]] for i, row in enumerate(matrix)]
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
                max_row = j
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
        factor = augmented_matrix[i][i]
        for j in range(i, n+1):
            augmented_matrix[i][j] /= factor
        for j in range(n):
            if j != i:
                factor = augmented_matrix[j][i]
                for k in range(i, n+1):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
    return [row[-1] for row in augmented_matrix]

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

# Define the 2-bit inner-product MetricGadget
def ip_2(v1, v2):
    return dot_product(v1[:2], v2[:2])

# Define the coordinatewise additive metric
def d_plus(v1, v2):
    return sum(hamming_distance(v1[i], v2[i]) for i in range(len(v1)))

# Define the GadgetComposition function
def gadget_composition(gadgets):
    n = len(gadgets)
    result = [[0] * (n**2) for _ in range(n**2)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                for l in range(n):
                    result[i*n+k][j*n+l] = gadgets[i][k] + gadgets[j][l]
    return result

# Define the LiftedInputSpace function
def lifted_input_space(gadget, metric):
    n = len(gadget)
    result = [[0] * (n**2) for _ in range(n**2)]

```

```

    for i in range(n):
        for j in range(n):
            for k in range(n):
                for l in range(n):
                    result[i*n+k][j*n+l] = metric(gadget[i], gadget[j]) + metric(gadget[k], gadget[l])
    return result

# Define the AsdimCertify function
def asdim_certify(lifted_space, R):
    n = int(math.sqrt(len(lifted_space)))
    for m in range(1, n+1):
        covers = []
        for i in range(n):
            for j in range(n):
                cover = [(i*n+k, j*n+l) for k in range(m) for l in range(m)]
                if all(d_plus(lifted_space[i*n+k], lifted_space[j*n+l]) <= R for (k1, l1), (k2, l2) in combinations(cover, 2)):
                    covers.append(cover)
        if len(covers) > 0:
            return m
    return n

# Define the CoarsePullbackProtocol function
def coarse_pullback_protocol(protocol, lifted_space):
    n = int(math.sqrt(len(lifted_space)))
    result = []
    for i in range(n):
        for j in range(n):
            result.append((i*n+j, protocol[i][j]))
    return result

# Define the run_trial function
def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Parameters
    k_values = [1, 2, 3]
    n_values = [2, 3]
    f_functions = [lambda x: all(x), lambda x: sum(x) % 2 == 0]

    metric_value = 0.0
    instances_tested = 0

    for k in k_values:
        G_k = gadget_composition([ip_2] * k)
        lifted_space = lifted_input_space(G_k, d_plus)

        for f in f_functions:
            Q_f = len(f(range(2**n)))

            # AsdimCertify
            m = asdim_certify(lifted_space, 4)
            if m < k + 1:
                return {
                    "metric_name": "AsdimCertify",
                    "metric_value": m,
                    "instances_tested": instances_tested,

```

```

        "conjecture_holds": False,
        "counterexample": f"asdim_certify failed for k={k}, Q(f)={Q_f}"
    }
    metric_value += m
    instances_tested += 1

    # Protocol enumeration
    max_cost = math.ceil(Q_f * math.log2(k + 1)) - 1
    found_protocol = False
    for c in range(1, max_cost + 1):
        protocol = [[0] * (n**2) for _ in range(n**2)]
        if enumerate_protocols(lifted_space, f, protocol, c):
            found_protocol = True
            break
    if not found_protocol:
        return {
            "metric_name": "ProtocolEnumeration",
            "metric_value": None,
            "instances_tested": instances_tested,
            "conjecture_holds": False,
            "counterexample": f"protocol enumeration failed for k={k}, Q(f)={Q_f}"
        }
    metric_value += c
    instances_tested += 1

    return {
        "metric_name": "AsdimCertify and ProtocolEnumeration",
        "metric_value": metric_value / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": True,
        "counterexample": ""
    }
}

# Define the protocol enumeration function
def enumerate_protocols(lifted_space, f, protocol, c):
    n = int(math.sqrt(len(lifted_space)))
    for i in range(n):
        for j in range(n):
            if sum(protocol[i*n+k][j*n+l] for k in range(n) for l in range(n)) != c:
                return False
    return True

# Main function to run trials and print results
if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    counterexample = next(result["counterexample"] for result in results if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_af1c65ed.py", line 185, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_af1c65ed.py", line 124, in run_trial
    G_k = gadget_composition([ip_2] * k)
          ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_af1c65ed.py", line 74, in gadget_composition
    result[i*n+k][j*n+l] = gadgets[i][k] + gadgets[j][l]
                          ~~~~~^
TypeError: 'function' object is not subscriptable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `gadget_composition` (treating a function as subscriptable) before any data was produced, so neither the support nor falsification criteria could be evaluated. | next: Fix the `gadget_composition` implementation to operate on metric matrices/arrays rather than function objects, then re-run the 12 (k,n,f) instances across 3 seeds.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	8784
2	novelty	claude_max	opus	0	0	3942
3	novelty	claude_max	opus	0	0	8916

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17902
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20406
6	judge	claude_max	opus	0	0	5461

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 65410 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/916caa07c776.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/916caa07c776.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/916caa07c776.tar.gz (if generated)